



**AMERICAN
UNIVERSITY
OF BEIRUT**

Module 1 – Lecture Notes

Mindset, Foundations & Prompt Engineering

AI-Assisted Coding

Module snapshot

This module teaches the habits behind reliable AI-assisted development. You will learn why AI-generated code can look correct while still being wrong, how to write prompts as compact technical specifications, and how to apply the loop: ask -> inspect -> run -> test -> refine.

By the end, you will have reviewed user stories, a one-paragraph architecture decision, a running FastAPI skeleton, and a short reflection log.

How to use these notes

Read these notes alongside the Module 1 videos. They are not a transcript. They are a student-facing guide to the concepts, examples, exercises, and deliverables taught in the module.

Learning outcomes

1. Explain, at a practical level, that AI coding assistants generate plausible outputs from patterns rather than verifying code at runtime.
2. Describe the main strengths of AI assistants: scaffolding speed, syntax/framework recall, and explaining unfamiliar code.
3. Identify three common failure modes: hallucinated APIs, missing non-functional requirements, and shallow business logic.
4. Apply the course mindset: AI is a junior collaborator; you are the senior developer who reviews, tests, corrects, and owns the result.
5. Define the Task Tracker project scope and keep excluded complexity out of Module 1 work.
6. Write structured prompts using role/context, task, constraints, and output format.
7. Use role prompting, few-shot prompting, and constraint-based prompting for practical software tasks.
8. Start AI conversations with a short context block and compare outputs from tools such as ChatGPT and Claude.

9. Apply the AI interaction loop: ask -> inspect -> run -> test -> refine.
10. Use AI to draft requirements, evaluate architecture options, and scaffold a minimal FastAPI project skeleton.

Module outputs

Output	What it should contain	Why it matters
Reviewed user stories	6-8 user stories with acceptance criteria and at least one documented correction to the AI draft.	Turns a vague project idea into buildable, testable requirements.
Architecture Decision Record (ADR)	One paragraph, in your own words, explaining the chosen architecture and assumptions you rejected.	Shows that AI proposed options, but you made the decision.
Running FastAPI skeleton	A minimal project structure with a /health endpoint returning HTTP 200 and Swagger docs at /docs.	Confirms that the plan has become executable code.
Reflection log	For each hands-on activity: one thing AI got right, one thing you corrected, and one assumption AI made.	Builds the habit of reviewing AI output rather than accepting it automatically.

Part 1 - Mindset and Course Context

Key takeaway

AI is a junior collaborator. It can be useful and fast, but it does not own the work. You are the senior developer who provides context, reviews every line, runs and tests the output, and takes responsibility for the final result.

Module 1 starts with mindset rather than tools or syntax. AI coding assistants are already part of modern software development. The important question is not whether to use them, but whether you can use them in a disciplined way that makes you faster and more reliable instead of sloppier.

Junior collaborator behavior	Senior developer behavior
Writes the happy path.	Gives clear project context before asking for output.
Forgets error handling or validation unless prompted.	Inspects every line before accepting code.
Misreads or fills in missing requirements.	Runs the generated code and observes actual behavior.
Does not know your conventions unless told.	Tests the result and refines the prompt or implementation.

Using AI vs. using AI well

Using AI casually	Using AI well
Copy-paste output.	Specify context and constraints.
Accept the first answer.	Treat the first answer as a draft.
Prompt instinctively.	Use a repeatable prompt structure.
Review only when something breaks.	Inspect deliberately, then run and test.

The skill gap is not access to a model. Two developers can use the same AI assistant and get very different outcomes. The difference is usually the quality of the specification, the review process, and the ability to turn a first draft into a better second draft.

Who this course is for

- Best fit: intermediate developers who can already build and ship applications.
- You will benefit if you use AI casually, accept output too quickly, or want a complete workflow from requirements through review.
- Not the target: complete beginners who still need programming fundamentals, or AI/ML researchers looking for model internals.

Part 2 - How AI Coding Assistants Work

Key takeaway

AI coding assistants are pattern-completion systems, not runtime-verification systems. They can generate code that looks plausible, but they do not know whether it works until you run it and provide evidence.

The module gives a practical mental model, not a deep theory lecture. When you submit a prompt, the assistant generates likely next tokens based on patterns it has learned. It is not automatically running your code, inspecting your terminal, reading your database, or checking your library versions.

This explains both the strength and the risk: when your task matches common patterns, the output can be fast and useful; when the pattern looks right but the detail is wrong, the output can be polished and still fail.

Three core strengths

Strength	How to use it deliberately
Speed of scaffolding	Generate boilerplate, file structures, repetitive patterns, and first-pass setup quickly.
Syntax and framework recall	Get help with APIs, framework conventions, and language details you do not want to look up manually.
Explaining unfamiliar code	Paste a function, stack trace, or confusing snippet and ask for a plain-language explanation, risks, and trade-offs.

Three core limitations

Limitation	What it means for you
No automatic project memory	Restate project context, stack, constraints, and decisions at the start of a new conversation.
No runtime context	The assistant cannot see your terminal, logs, database, environment variables, or failures unless you provide them.
No reliable uncertainty signal	Correct and incorrect answers can sound equally confident. Tone is not evidence.

Failure mode 1: hallucinated APIs

A hallucinated API is a method, module, or function that looks like it should exist but does not. The method name often follows the style of the library, so the code can look normal during a quick review.

```
# AI-generated and plausible-looking
from pydantic import BaseModel

class Task(BaseModel):
    title: str

Task.validate_fields(data)

# Problem: validate_fields() does not exist.
```

Review question

Does this method actually exist in the library version I am using?

Failure mode 2: missing non-functional requirements

If you ask for a basic endpoint, AI often writes the happy path: accept input, create an object, and return a generic response. The missing pieces are usually validation, response models, status codes, structured errors, and edge cases.

Vague POST /tasks output	Production-aware POST /tasks output
Accepts JSON.	Validates title and allowed enum values.
Creates a task.	Uses a Pydantic response schema.
Returns generic 200 status.	Returns 422 with a useful detail when input is invalid.
Leaves scope implicit.	Keeps scope explicit and does not add unwanted features.

In this module, "production-aware" means you build the habit of thinking about validation, errors, and edge cases. It does not mean Module 1 is producing production-grade software.

Failure mode 3: shallow business logic

AI can write syntactically correct setter code without discovering the real rules behind the update. For Task Tracker status updates, shallow code might simply assign whatever new status was provided.

- Which statuses are valid?
- Can a task in Done move back to ToDo or InProgress?
- What happens if the task does not exist?
- What happens if the same update is sent twice?

Takeaway

The answer is not to avoid AI. The answer is to review deliberately. Verify, do not hope.

Exercise: spot the flaw

For each snippet below, identify the failure mode and explain the risk in one sentence.

Snippet	Question to ask
<code>express.validateSchema(req.body, taskSchema)</code>	Does this API exist?
<code>PATCH /tasks/:id</code> updates any field	Where are the status rules?
<code>DELETE</code> returns 200 even when id is missing	What should the API return?

Part 3 - Prompts as Specifications

Key takeaway

A prompt is a compact technical specification, not a search query or casual request. Better specifications usually produce better AI output.

The module shifts from understanding AI behavior to controlling AI behavior. A prompt should define the same things you would clarify for another developer: context, exact task, boundaries, and expected output shape.

The four-part anatomy of a good prompt

Part	Purpose	Example
Role / context	Sets the lens and project situation.	You are a senior FastAPI developer reviewing production readiness.
Task	Defines the work to do.	Generate, refactor, validate, review, or explain a specific artifact.
Constraints	Defines boundaries and negative constraints.	No auth. No external libraries. Return 422 for invalid input.
Output format	Defines the shape of the answer.	Return only code in one block. No explanation.

Anatomy in action: weak to strong

Weak prompt	Structured prompt
Generate a user login function.	You are a senior backend developer building a FastAPI API. Write a POST <code>/login</code> endpoint. Requirements: accept email and password in JSON; return 400 if either field is missing; return 401 if credentials are invalid; do not implement user lookup. Return only the endpoint function.

What changed: the role shapes the perspective, the task removes guessing, the constraints prevent hallucinated project logic, and the output format makes the result usable.

Four common prompt mistakes

Mistake	Better habit
Assuming the assistant knows your stack or codebase.	State the language, framework, version when relevant, and project conventions explicitly.
Asking for too many things at once.	Split authentication, tests, docs, refactoring, and debugging into separate prompts.
Using vague verbs such as fix, improve, or clean up.	Say what improvement means: safer, faster, shorter, more readable, or more defensive.
Omitting negative constraints.	Say what the output must not include: no auth, no logging, no new libraries, no extra prose.

The absence of a constraint is often treated as permission. If you do not want a database, authentication, Docker, logging, or extra libraries, say so.

Iterative prompting: the first response is a draft

A weak workflow treats the first answer as binary: accept it or throw it away. A better workflow treats the first answer as a draft. Inspect it, keep what works, identify what is missing, and write a targeted follow-up.

Targeted follow-up example

The endpoint accepted a whitespace-only title. Add validation that strips whitespace before checking emptiness and returns 422 with a clear error message. Keep the existing function signature.

Exercise: rewrite the prompt

Rewrite each weak prompt using role/context, task, constraints, and output format.

- Weak prompt 1: "Fix the bug in my task filtering code."
- Weak prompt 2: "Write me some tests for the API."

For the first prompt, include the stack, expected behavior, actual behavior, and relevant code. For the second prompt, include the test framework, endpoints or scenarios, constraints, and output format.

Part 4 - Prompt Engineering Patterns and the AI Interaction Loop

Key takeaway

Prompt engineering is not magic wording. It is a learnable craft built from a few practical patterns used consistently.

The module teaches three reusable patterns: role prompting, few-shot prompting, and constraint-based prompting. In real work, they are often combined.

Pattern 1: role prompting

Role prompting assigns a specific professional lens. Avoid vague roles such as "expert". Use roles that define what the assistant should pay attention to.

You are a senior FastAPI developer reviewing a REST API for production readiness.

Review the following endpoint and identify issues related to error handling, input validation, and HTTP status codes.

[paste endpoint code]

Pattern 2: few-shot prompting

Few-shot prompting gives the assistant one or more examples so it can imitate the structure and quality level you want. The example sets the quality ceiling, not just the format.

Generate user stories for the Task Tracker in this format.

Example:

Story: As a team member, I want to create a task so that I can track work.

Acceptance Criteria:

- Title is required; missing title returns 422.
- Description is optional.
- Created task appears in the list.

Now generate 5 more stories in the same format.

Best uses: user stories, test cases, ADRs, API docs, and commit messages.

Pattern 3: constraint-based prompting

Constraint-based prompting lists what the output must and must not include. It is the most direct way to prevent scope creep, unexpected dependencies, and wrong output shapes.

Write a function to validate a task object before saving it.

Constraints:

- Use only Python standard library and Pydantic.
- Do not add logging.
- Return: { valid: bool, errors: list[str] }.
- Title must be non-empty after stripping whitespace.
- Status must be ToDo, InProgress, or Done.
- Priority must be Low, Medium, or High.

Return only the function. No explanation.

When to use each pattern

Pattern	Use when...	Example course use
Role prompting	A specific professional perspective is needed.	Review an API endpoint for validation, errors, and status codes.
Few-shot prompting	The desired structure or quality is easier to show than describe.	Generate user stories matching a strong example.
Constraint-based prompting	You need to prevent extras, scope creep, dependencies, or wrong return shapes.	Scaffold only the skeleton; do not add auth, database, or full CRUD.
All three together	The output is complex or high-stakes.	Architecture analysis, ADR drafts, security reports, structured test cases.

The AI interaction loop

Core workflow

Do not submit and accept. Use the loop: ask -> inspect -> run -> test -> refine.

Step	What it means	What it looks like in practice
Ask	Write a clear structured prompt.	Include role/context, task, constraints, and output format.
Inspect	Read the output before using it.	Check imports, function names, assumptions, error cases, and return shape.
Run	Execute the code or apply the artifact.	Start the server, paste the function into the project, or review the requirement for buildability.
Test	Verify behavior explicitly.	Use test cases, curl, Swagger, or acceptance criteria. "It looked right" is not a test.
Refine	Feed the smallest correction back.	Use the exact error, failing case, or missing requirement as the next prompt.

The loop also applies to non-code artifacts. For requirements, "run" means checking whether the story can actually be built and tested. For architecture, it means evaluating trade-offs before accepting a proposal.

Loop example: Task Tracker validation

Step	Example
Ask	Write a validation function for title, status, and priority. Return { valid, errors }.
Inspect	Does it trim whitespace? Does it use the exact allowed values? Does the return shape match?
Run	Paste it into the project and check imports and execution.
Test	Try empty title, whitespace-only title, invalid status, and a valid object.
Refine	If whitespace-only title is accepted, ask for a trim check and exact error message.

Exercise: apply all three patterns

Generate user stories for a Task Tracker that supports priorities and assignees. Write three separate prompts: one role prompt, one few-shot prompt, and one constraint-based prompt. Submit them to your AI tool and compare the outputs for structure, acceptance criteria, and edge cases.

Part 5 - Define the Project: Task Tracker

Key takeaway

The Task Tracker is intentionally simple so the AI-assisted workflow stays visible. Simple project, serious workflow.

Before the hands-on work, the module creates a shared understanding of the project. The finished application is a simple task board backed by a FastAPI API and a simple web frontend. Module 1 does not build the complete app; it gets the foundation running.

What the finished project tour shows

- Task board with To Do, In Progress, and Done columns.
- Task cards with title, description, priority, assignee, and status.
- New Task modal or create form.
- Edit flow for changing task details and status.
- Filtering by status and priority.
- Delete action.
- Swagger API documentation at /docs.

Task Tracker scope

Included	Explicitly excluded
Create, view, update, and delete tasks.	Authentication and user accounts.
Filter by status and priority.	Multi-tenancy and per-user task lists.
Fields: id, title, description, status, priority, assignee.	Real-time updates, mobile app, and notifications.

Included	Explicitly excluded
Swagger API docs through FastAPI.	Production database and deployment in Module 1.

The excluded features are deliberate. Authentication, multi-tenancy, real-time sync, and mobile support would hide the workflow behind product complexity. The course focuses first on habits that transfer to larger projects later.

Part 6 - Define the Tool: ChatGPT, Claude, and Transferable Habits

Key takeaway

The tool changes; the habits do not. Prompt anatomy, prompt patterns, the stateless mental model, and the interaction loop work across capable AI assistants.

The tool section focuses mainly on ChatGPT and Claude. The same habits transfer to Gemini, Copilot, and other assistants, but Module 1 uses browser-based tools so the workflow stays visible.

Why compare two tools?

Tool	Strengths emphasized in the lecture
ChatGPT	General-purpose coding, conversational follow-ups, exploratory explanations, and quick iteration.
Claude	Constraint following, long context, structured outputs, and course scaffolding. Claude is the reference tool for the scaffold demo.

Using more than one tool helps you notice assumptions and blind spots. It also prevents tool dependency. Your goal is not to memorize one interface; your goal is to build transferable habits.

The stateless mental model

Every new conversation starts from zero unless you provide context or work inside a persistent project/workspace. The assistant does not reliably know your project, decisions, constraints, or previous corrections unless you restate them.

Reusable context block

I am building a Task Tracker REST API using Python and FastAPI.

The app supports create, view, update, and delete tasks.

Each task has id, title, description, status, priority, and assignee.

No authentication and no external database yet.

I am using the loop: ask, inspect, run, test, refine.

Interface elements developers should notice

Element	Why it matters
Message bar / message area	Use multi-line prompts. Structure the prompt instead of writing one dense paragraph.
Model selector	Choose a strong coding-capable model for development tasks.
New chat / new conversation	Avoid context contamination between unrelated tasks.
Code copy button	Copy complete code blocks without formatting mistakes.
Custom Instructions / Projects	Useful for persistent preferences or project context, but not a replacement for task-specific context.

Reasoning modes: when to use deeper thinking

Use thinking/reasoning when...	Skip it when...
Evaluating architecture trade-offs.	Generating simple boilerplate.
Reasoning through status transition rules.	Creating a basic .gitignore.
Debugging unclear errors.	Looking up simple syntax or commands.
Designing validation chains or business logic.	Writing straightforward setup text.

Practical rule: judgment = think; recall = skip.

Exercise: First Contact

Submit the same prompt to two tools and compare their answers.

I am building a simple Task Tracker REST API in Python and FastAPI.

The API supports CRUD operations for tasks.

Each task has id, title, description, status, priority, and assignee.

List the five most important technical decisions I should make before writing any code.

For each decision, explain why it matters and suggest a default for a small learning project.

- Which tool was more structured?
- Which answer was more actionable?
- What assumptions did each tool make, such as database, authentication, Docker, or frontend framework?

Part 7 - Hands-On Session: Requirements, Architecture, Scaffold

Key takeaway

Each output becomes context for the next prompt: requirements -> architecture -> scaffold. This chained workflow keeps AI-assisted development coherent instead of random.

Part	Activity	Output
A	AI-assisted requirements	6-8 reviewed user stories with acceptance criteria.
B	AI-assisted architecture and tech stack	One-paragraph ADR in your own words.
C	Scaffold project skeleton	Running FastAPI skeleton with /health returning 200 and Swagger at /docs.

Part A: AI-assisted requirements

A project idea such as "build a task tracker" is not a specification. It hides unanswered questions: what statuses are allowed, what makes a title valid, what happens when a task is missing, and whether status transitions are restricted.

AI is useful here because it removes the blank page. It can draft user stories quickly, but the AI draft is not the deliverable. Your job is to review, correct, and own the requirements.

Expected story area	What to check
Create task	Title is required; description can be optional; created task appears in the list.
View tasks	All tasks are returned in a predictable shape.
Filter tasks	Filtering by status and priority behaves clearly and rejects invalid filter values.
Update task details	Allowed fields are clear; invalid input is handled.
Move through statuses	Allowed transitions are explicit. The lecture correction highlights that Done should not move back to ToDo or InProgress.
Delete task	Missing task behavior and status code are defined.
Validate task data	Empty or whitespace-only title, invalid status, and invalid priority are rejected.

Review focus

Ask: Is each story buildable? Is each acceptance criterion testable? Did AI add auth, users, admin roles, or other out-of-scope assumptions? Did it miss edge cases such as whitespace-only title or invalid status transitions?

Part B: AI-assisted architecture and tech stack

After the requirements are reviewed, use them as context for architecture. AI is good at exploring options quickly, but it is not the decision maker. The ADR must be written in your own words.

Architecture option discussed	Trade-off
FastAPI + JSON file storage	Very simple and good for a first learning project, but less realistic for larger backends.
FastAPI + SQLite / SQLAlchemy	More realistic backend storage and query behavior, but adds complexity earlier.

Evaluation criterion	Question to ask
Simplicity	Can you understand and maintain this structure alone?
Testability	Does the structure make tests easy to write?
Local run / deployability	Can you run it locally with one or two commands?
Familiarity	Do you already know the tools, or can you learn them quickly?

Assumptions to catch include authentication, Docker, cloud deployment, microservices, heavy ORMs, Redis/Celery, or frontend frameworks that were not requested.

ADR requirement

Write one paragraph in your own words. Include the chosen architecture, why it fits the four criteria, at least two AI assumptions you rejected, and one risk you would revisit if the project grew.

Part C: scaffold the project skeleton

This is the first real code you run. The goal is not the full Task Tracker app. The goal is a minimal FastAPI foundation with one working endpoint.

Skeleton should include	Skeleton should not include yet
Folders such as app/, app/api/routes/, app/core/, app/models/, app/storage/, and tests/.	Full CRUD task endpoints.
requirements.txt with FastAPI/Uvicorn dependencies.	Authentication or user accounts.
app/main.py creating the FastAPI app.	A production database or deployment setup.
GET /health returning HTTP 200 with JSON such as {"status": "ok", "timestamp": "..."}.	Full business logic or data persistence behavior.
.env.example, .gitignore, and README.md.	Docker/cloud/microservice complexity in Module 1.

Apply the full loop: ask for the scaffold with your ADR as context, inspect every file, run the server, test /health, and refine using exact errors when something fails.

```
# Typical run and verification flow
python -m venv venv
# activate the environment for your OS
pip install -r requirements.txt
uvicorn app.main:app --reload
curl http://localhost:8000/health
# Then open http://localhost:8000/docs in the browser.
```